

Deep Learning Systems Analysis Report

Time-Series Demand Forecasting for LLM Inference Serving

Project: Deep Learning Systems

Date: April 2026

Author: Sambasiva Andaluri

Report Overview

This project studies predictive demand forecasting for Large Language Model (LLM) inference services using deep learning methods. Using the Azure LLM Inference Trace 2024 dataset, which contains production traces from Microsoft Azure’s code generation and conversation services, we built and compared LSTM encoder-decoder and Transformer architectures for predicting request arrival rates up to 60 minutes into the future. The goal is to evaluate how well these models capture short-term demand variation and whether they offer meaningful improvement over simpler baselines.

Dataset and Task Description

Dataset: Azure LLM Inference Trace 2024

The dataset contains production traces from two LLM inference services operating on Microsoft Azure infrastructure over approximately one week in May 2024. The traces were published with the HPCA '25 paper “DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency” by Stojkovic et al. (2025).

Data Summary:	Service	Total Requests	Duration	Avg Requests/5min
Code Generation	~16.8 million			
~1 week | ~5,800 | | Conversation | ~27.3 million | ~1 week | ~9,500 |

Schema: - **TIMESTAMP:** Request arrival time (datetime) - **ContextTokens:** Number of input/context tokens - **GeneratedTokens:** Number of output tokens generated

Task: Multi-Step Demand Forecasting

Objective: Predict the request arrival rate (requests per 5-minute bin) for the next H timesteps, given the previous T timesteps of historical data.

Configuration: - Time granularity: 5-minute bins - Lookback window (T): 24 steps = 2 hours - Forecast horizon (H): 12 steps = 1 hour - Features: request_count, mean_context_tokens, mean_generated_tokens

Why This Matters: LLM inference is computationally intensive and highly sensitive to changes in request volume. Forecasting short-term demand can

support capacity planning, help characterize recurring traffic structure, and provide a basis for evaluating predictive scaling strategies in a controlled setting.

Model Architecture and Design Decisions

Model A: LSTM Encoder-Decoder

Architecture:

LSTM Encoder-Decoder Architecture:

```
LSTMEncoderDecoder(  
    (encoder): LSTM(3, 64, num_layers=2, batch_first=True, dropout=0.2)  
    (decoder): LSTM(1, 64, num_layers=2, batch_first=True, dropout=0.2)  
    (fc): Linear(in_features=64, out_features=1, bias=True)  
    (dropout): Dropout(p=0.2, inplace=False)  
)
```

Total parameters: 101,441

Design Rationale: - Sequence-to-sequence architecture naturally handles variable-length input/output (Sutskever et al., 2014) - LSTM cells capture long-range dependencies critical for diurnal patterns - Guided training (teacher forcing) during training accelerates convergence - Dropout (0.2) between layers prevents overfitting

Hyperparameters: - Hidden dimension: 64 - Number of layers: 2 - Dropout: 0.2 - Guided training ratio: 0.5 - Optimizer: Adam (lr=1e-3) - Scheduler: ReduceLROnPlateau (factor=0.5, patience=5) - Early stopping patience: 10 epochs

Model B: Transformer Encoder

Architecture:

Transformer Forecaster Architecture:

```
TransformerForecaster(  
    (input_projection): Linear(in_features=3, out_features=64, bias=True)  
    (pos_encoder): PositionalEncoding(  
        (dropout): Dropout(p=0.1, inplace=False)  
    )  
    (transformer_encoder): TransformerEncoder(  
        (layers): ModuleList(  
            (0-1): 2 x TransformerEncoderLayer(  
                (self_attn): MultiheadAttention(  
                    (out_proj): NonDynamicallyQuantizableLinear(in_features=64, out_features=64, bias=True)  
                )  
                (linear1): Linear(in_features=64, out_features=128, bias=True)  
                (dropout): Dropout(p=0.1, inplace=False)
```

```

        (linear2): Linear(in_features=128, out_features=64, bias=True)
        (norm1): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
        (norm2): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
        (dropout1): Dropout(p=0.1, inplace=False)
        (dropout2): Dropout(p=0.1, inplace=False)
    )
)
(output_projection): Linear(in_features=64, out_features=12, bias=True)
)

```

Total parameters: 67,980

Design Rationale: - Self-attention can capture dependencies at any distance without sequential processing (Vaswani et al., 2017) - Parallel computation enables faster training on GPU - Positional encoding preserves temporal order information - Direct projection to horizon avoids autoregressive error accumulation

Hyperparameters: - `d_model`: 64 - Attention heads: 4 - Feedforward dimension: 128 - Dropout: 0.1 - Same optimizer and scheduler as LSTM

Baseline Models

1. **Naive Forecast:** Predict last observed value for all future steps
2. **Seasonal Naive:** Predict value from same time yesterday (lag-288)
3. **Linear Regression:** Fit separate linear models for each horizon step using lag features

Experimental Comparison

Experimental Design

Control Variable: Model architecture (LSTM vs Transformer)

Fixed Variables: - Dataset (Code service, 5-min granularity) - Train/val/test split (70/15/15%, time-based) - Lookback window (24 steps) - Forecast horizon (12 steps) - Optimizer settings (Adam, lr=1e-3) - Early stopping (patience=10) - Batch size (32) - Random seed (42)

Why Code Service Only: - Controlled comparison requires identical data for both architectures - Code service has higher variance (burstier), making it the harder test case - Conversation service reserved for cross-service transfer evaluation

Evaluation Dimensions: 1. Prediction accuracy (MAE, RMSE, R^2) 2. Error degradation with horizon 3. Diurnal pattern capture 4. Computational efficiency 5. Cross-service generalization

Why This Comparison is Meaningful

The LSTM and Transformer represent fundamentally different approaches to sequence modeling: - LSTM processes sequences recurrently, maintaining state through time - Transformer uses attention to relate all positions simultaneously

This comparison tests whether attention-based parallelism improves forecasting over recurrent processing for demand prediction.

Results and Interpretation

Overall Performance Comparison

Model	MAE	RMSE	R ²	Training Time (s)	Inference (ms)
Naive	1,350	1,961	0.935	0	0
Seasonal Naive	1,308	1,925	0.937	0	0
Linear Regression	1,090	1,528	0.960	<1	<1
LSTM Encoder-Decoder	1,254	1,756	0.948	30.1	0.44
Transformer	1,050	1,504	0.962	12.0	0.81

Key Findings

1. **Transformer achieves best performance:** Lowest MAE (1,050 requests/5min), lowest RMSE (1,504), and highest R² (0.962). It outperforms all baselines and LSTM.
2. **LSTM underperforms expectations:** With MAE=1,254, LSTM is actually worse than Linear Regression (MAE=1,090). The autoregressive decoder’s error accumulation likely hurts performance—small errors in early steps compound through later predictions.
3. **Linear Regression is surprisingly competitive:** At R²=0.960, simple linear combinations of lag features capture 96% of variance. This suggests the demand patterns are largely linear, with deep learning providing only marginal improvement.
4. **Seasonal Naive validates diurnal patterns:** Beating the Naive baseline by 42 MAE confirms the strong 24-hour periodicity identified in our autocorrelation analysis.
5. **Transformer trains 2.5x faster:** 12 seconds vs 30 seconds, due to parallel attention computation versus sequential LSTM processing.

Performance Trade-offs

LSTM Characteristics: - Autoregressive decoding compounds errors over forecast horizon - Guided training (teacher forcing) stabilizes training but creates train/inference mismatch - Slower training due to sequential computation

Transformer Characteristics: - Direct projection to all 12 steps avoids error accumulation - Faster training through parallelization - Slightly slower inference (0.81ms vs 0.44ms) but still sub-millisecond

Operational Evaluation

Predictive Autoscaling Performance (90th Percentile Threshold):

Metric	LSTM	Transformer
Precision	1.000	0.912
Recall	0.694	0.861
F1 Score	0.820	0.886
False Alarm Rate	0.000	0.013
Avg Advance Warning (min)	0.625	0.926

These results show that both models can identify high-demand periods under the threshold-based simulation, with the Transformer providing the better balance of precision and recall. However, the average advance warning is less than one minute for both models. The experiment is therefore most useful as a comparative evaluation of forecast-informed thresholding rather than evidence of deployment-ready autoscaling performance.

Cross-Service Transfer

Training	Testing	MAE	R ²	Interpretation
Code	Code	1,050	0.962	In-domain baseline
Code	Conversation	2,202	-1.743	Severe degradation

For the LSTM, Code-to-Conversation transfer also performed poorly (MAE 2,131; R² -1.958). These results indicate that models trained on the Code service do not generalize well to the Conversation service without adaptation. The evidence in this project therefore supports service-specific modeling rather than direct cross-service reuse.

Limitations and Risks

Dataset Limitations

- **Single infrastructure:** Results may not generalize to non-Azure deployments
- **Trace duration:** The analysis uses approximately one week of data and may not capture longer-term weekly or monthly structure
- **No external features:** Excludes events, holidays, or promotional periods that drive demand

Architecture Limitations

- **Fixed horizon:** Models predict exactly 12 steps; different horizons require retraining
- **Univariate focus:** Primarily predicts request count; token counts used as features only
- **No uncertainty quantification:** Point predictions without confidence intervals

Operational Risks

- **False positives:** Threshold-triggered decisions can still produce unnecessary scale-up actions
- **False negatives:** Missed spikes remain possible, particularly under abrupt demand changes
- **Distribution shift:** Model performance degrades if traffic patterns change over time

Potential Failure Cases

- **Flash crowds:** Sudden viral events cause demand spikes without historical precedent
- **Infrastructure changes:** Model updates or service migrations alter patterns
- **Adversarial traffic:** Attack patterns differ from organic demand

Ethical and Responsible Use

Energy Consumption

Deep learning training requires compute resources, although the models used here are modest in size relative to many modern sequence models. In this project, the LSTM has 101,441 parameters and the Transformer has 67,980 parameters. If similar models were deployed at scale or retrained frequently, the cumulative energy cost would still warrant attention. Carbon-aware scheduling and efficient retraining policies would therefore be sensible considerations.

Bias Considerations

The Azure dataset reflects usage patterns of a specific user population (primarily developers and enterprise users). Models trained on this data may not accurately forecast demand from different demographics or regions.

Privacy

While the dataset contains only aggregate counts (not user content), temporal patterns could potentially reveal business-sensitive information about Azure customers' usage patterns.

Dual-Use Concerns

Accurate demand forecasting could be misused for: - Timing attacks to overload services during predicted low-capacity periods - Competitive intelligence about service popularity

Mitigation: Deploy models within trusted infrastructure; avoid publishing real-time predictions externally.

Future Improvements

Short-Term (Additional Resources)

1. **Ensemble methods:** Combine LSTM and Transformer predictions to reduce variance
2. **Uncertainty quantification:** Add probabilistic forecasting (e.g., quantile regression)
3. **Multi-horizon training:** Train single model for multiple forecast horizons

Medium-Term (Additional Data)

1. **External features:** Incorporate time-of-day encodings, day-of-week, holidays
2. **Cross-service learning:** Multi-task learning across code and conversation services
3. **Longer history:** Extend training data to capture weekly patterns

Long-Term (Architecture Improvements)

1. **Temporal Fusion Transformer:** State-of-the-art architecture for multi-horizon forecasting (Lim et al., 2021)
2. **Neural ODE:** Continuous-time modeling for irregular timestamps
3. **Federated learning:** Train across multiple datacenters while preserving privacy

Connection to Synthesis

This demand forecaster can be viewed as one component of a broader infrastructure management workflow: - **Orchestration logic** could use forecasts to inform capacity adjustments - **Cost-aware policies** could weigh forecast confidence against provisioning cost - **Service-level monitoring** could be used to recalibrate thresholds over time

Conclusion

Interpretation for a Non-Technical Audience

For a non-technical reader, the project asks a simple question: can we estimate how busy an AI service will be in the next hour by looking at its recent traffic?

If the answer is yes, operators could prepare computing resources earlier instead of reacting only after demand spikes occur. In this study, the Transformer model produced the strongest overall results, but the improvement over a simpler Linear Regression baseline was modest. The main practical takeaway is that short-term demand in these traces is predictable, but careful baseline comparisons are essential because simpler models already capture much of the useful signal.

On Model Complexity vs. Accuracy

The Transformer achieved only marginal improvement over Linear Regression (MAE: 1,050 vs 1,090, a 3.7% reduction). This finding suggests the Azure LLM demand patterns are largely linear and well-captured by simple lag relationships. The strong 24-hour autocorrelation we observed means that “same time yesterday” provides most of the predictive signal, which linear models exploit effectively.

Deep learning’s value in this context lies not in dramatic accuracy gains but in:

1. **Potential for automatic feature discovery** with longer training periods or more diverse data
2. **Handling of edge cases** (demand spikes, anomalies) not fully visible in aggregate metrics
3. **Extensibility** to multivariate forecasting, multi-service learning, or real-time adaptation

Practical Recommendation

For this dataset, the main takeaway is that strong baselines are essential. Linear Regression performs competitively, and the Transformer provides a modest improvement in accuracy with acceptable training cost. If the goal is an exploratory forecasting study or a controlled model comparison, the Transformer is the strongest model evaluated here. If the goal is operational deployment, additional validation would be needed, including recalibration over time, uncertainty estimation, and a backtest tied to explicit provisioning objectives.

Role of Feature Engineering

This comparison used minimal feature engineering—raw time series with basic aggregations. Both model types received identical inputs. Future work could explore explicit time-of-day encodings, lag-288 features (same time yesterday), or rolling statistics. Such engineering might further close the gap between linear and deep learning approaches, or alternatively provide deep learning with richer building blocks for discovering complex interactions.

References

- Bergmeir, C., & Benítez, J. M. (2012). On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191, 192-213.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780.

- Lim, B., Arik, S. Ö., Loeff, N., & Pfister, T. (2021). Temporal Fusion Transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4), 1748-1764.
- Stojkovic, U., et al. (2025). DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency. *IEEE International Symposium on High-Performance Computer Architecture (HPCA '25)*.
- Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. *Advances in Neural Information Processing Systems*, 27.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.